TST: A Structured Memory Kernel for Edge Language Models

Aditya Yadav

Independent Research

github.com/Officially-aditya/TST

March 2026

Abstract

TST is a three-tier structured memory kernel for sub-1B language models on consumer edge hardware, comprising Short-Term Memory (STM, ring buffer), Long-Term Memory (LTM, Ternary Search Trie), and Tree Memory a directed acyclic graph that indexes project structure as a dependency-aware retrieval index. The Rust kernel achieves 137,000 read operations per second at <0.01 ms average latency within a 23 MB footprint for 200K symbols, validated across 36 stress tests. Tree Memory scales to 110K nodes (10K-file monorepo equivalent) with 19 ms build time and 11.4 MB resident; focused single-file dependency queries remain at 2–3 ms independent of project size. A FunctionGemma-270M/Qwen3.5-0.8B tiered router achieves 100% routing accuracy at 1,350 ms warm latency; we document a previously undescribed FunctionGemma silent failure under `float16` on Apple MPS and resolve it with no model changes.

To characterize retrieval selectivity, we conduct a 3-run ablation over a 24-bug, 10-file, 4-tier benchmark. Cross-file Tier 2 bugs are correctly detected when import-graph-scoped context (∼150 tokens, direct dependencies only) is injected, confirming that Tree Memory's subgraph retrieval conveys actionable cross-file information. The ablation further reveals a selectivity boundary: bulk injection (∼500–800 tokens) collapses performance below the no-context level, and non-native evidence formats collapse independently of token count. Tier 3 and Tier 4 bugs remain undetected at 0.8B scale a documented model-capacity limit, not an architectural one. The Tree Memory store and retrieval interface are model-agnostic and ready for larger consumers.

1 Introduction

Edge AI deployment of small language models (SLMs, 0.5–8B parameters) faces a structural limitation: context windows are consumed by conversation history, leaving no budget for persistent cross-session knowledge. Retrieval-augmented generation (RAG) (Lewis et al., 2020) addresses factual knowledge but is poorly suited for the fine-grained, low-latency, token-level context that code analysis, preference retention, and multi-file reasoning require. Systems like MemGPT (Packer et al., 2023) operate at page granularity and require large-model backends. There is no existing system that provides sub-millisecond structured memory with a hardware footprint compatible with edge deployment on devices without data-center GPUs.

TST addresses this gap for three concrete deployment scenarios: on-device code assistants that maintain project structure and cross-file dependency knowledge across sessions without cloud round-trips; edge-deployed personal assistants that persist user preferences in a weight-ranked retrieval index within a single-digit MB budget; and IoT or embedded systems where the full memory kernel must fit alongside application code in tens of megabytes.

We make the following contributions:

1. **TST Memory Kernel**: a three-tier Rust kernel (STM ring buffer, LTM Ternary Search Trie, Tree DAG) with 24-byte nodes and cache-friendly sequential layout achieving 137,000 read operations per second at <0.01 ms average latency on Apple M-series hardware, within a 23 MB footprint for 200K symbols. All 36 stress tests pass on production hardware.

2. **Retrieval Selectivity Characterization**: a 3-run ablation establishing that import-graph-scoped retrieval (∼150 tokens, direct dependencies only) is the correct operating point for 0.8B consumers. Bulk retrieval (>500 tokens) and non-native evidence formats both collapse performance independently. Tier 2 cross-file detections are confirmed under targeted injection; Tier 3 and 4 failures are documented as model-capacity limits. The store and retrieval interface are ready for larger consumers.

3. **Tree Memory as a Structural Context Store**: a dependency-aware DAG that indexes project structure (File, Function, Class nodes with typed import edges) and exposes O(depth)-bounded subgraph queries. The store scales to 110K nodes (10K-file monorepo) in 11.4 MB with 2–3 ms focused queries, independent of project size. It is

model-agnostic: any consumer queries the same subgraph API.

4. **Tiered SLM Router**: FunctionGemma-270M (Tier 1) → Qwen3.5-0.8B (Tier 2) pipeline reaching 100% routing accuracy at 1,350 ms warm latency an 8.7× improvement from the Qwen-only baseline achieved by precision fix and tool description tuning, with no model retraining.

5. **FunctionGemma Float16 Silent Failure**: documentation and diagnosis of a previously undescribed failure mode where RecurrentGemma SSM layers silently produce all `<pad>` output under `float16` on Apple MPS. Confirmed on all devices without `bfloat16` support; `float32` is the only workaround.

2 System Architecture

2.1 Overview

Figure 1 shows the full pipeline. User input enters an *Interpreter* that extracts intent and optional file paths. A *Tiered Router* classifies the query into one of four memory routes: STM (recent conversational context), LTM (persistent preferences and facts), Tree (code structure and dependency graph), or Cloud (general knowledge with no local memory). The Rust *Kernel* executes memory reads and writes atomically via an RWLock-protected guard. A *Worker SLM* (Qwen3.5-0.8B) generates the response conditioned on memory context. Logit bias from the kernel is optionally injected at generation time.

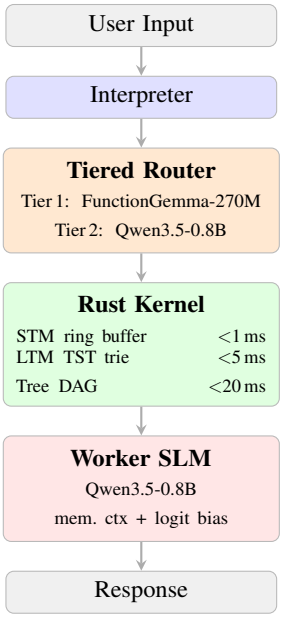

Figure 1: TST Memory System pipeline. The Tiered Router classifies input into one of four memory routes (STM, LTM, Tree, Cloud). The Rust Kernel executes reads and writes atomically; the Worker SLM generates responses conditioned on retrieved context and logit bias.

2.2 Memory Tiers

Short-Term Memory (STM). A 256-slot ring buffer indexed by key hash. Reads and writes are O(1). Entries carry a `decay_score` updated per access as $s_{t+1} = \alpha \cdot s_t + (1-\alpha)$ with $\alpha = 0.98$. When a score exceeds a promotion threshold, the entry is promoted to LTM. The ring buffer is designed for transient conversational tokens; it is intentionally not persisted across sessions.

Long-Term Memory (LTM). A Ternary Search Trie (Bentley and Sedgewick, 1997) backed by an Arena allocator. Each node is exactly 24 bytes with a cache-friendly sequential layout, storing two ASCII boundary bytes, a middle-child pointer, left/right sibling pointers, and a payload index. Payload types include `TokenStats` (frequency, canonical form, decay score) and `Preference` (key, value, weight). `Preference` payloads are designed for cross-session storage; `TokenStats` support logit bias computation.

Tree Memory (DAG). A structural context store implemented as a directed acyclic graph of `TreeNode` objects. Node types are Project, File, Function, and Class; edges represent import dependencies via `DependencyChanged` events. The key property is *dependency-aware retrieval*: given any File node, a `TREE_QUERY(node_id, depth)` call returns exactly the subgraph reachable via its import edges the function signatures of everything the file depends on, and nothing else. This is structurally equivalent to an import-graph-scoped retrieval index.

The store is model-agnostic: it holds typed structural facts about a codebase and exposes them via bounded subgraph queries. Any consumer a 0.8B model, an 8B model, a static analyzer can query the same store with the same API. The kernel exposes STDIO commands `TREE_INSERT`, `TREE_LINK`, `TREE_QUERY`, and `TREE_CLEAR`. The CLI populates the store from source files and queries it to construct per-file context for the Worker SLM.

2.3 Logit Bias

The kernel computes token-level logit bias as:

$$b(f,s) = \text{clamp}(\ln(1+f) \cdot s,\ b_{\min},\ b_{\max}) \tag{1}$$

where f is token frequency and s is the decay score. Non-finite inputs ($\pm\infty$, `NaN`) return $b = 0$. The formula naturally saturates at $\ln(1+f) \cdot 1.0 \approx 4.605$ for $f \geq 100$, making the clamp self-limiting even at aggressive bounds of ± 10.

Table 1: FunctionGemma precision diagnostic on Apple MPS.

dtype	Device	Output
`float32`	MPS	Correct `<start_function_call>` output
`float16`	MPS	All `<pad>` (token ID 0), silent
`float16`	CPU	All `<pad>` (token ID 0), silent

2.4 Persistence

LTM persistence uses snapshot-based serialization: the full Arena and TST are serialized via `serde_json` and written atomically (temp file + rename). STM is not persisted (transient by design). Tree Memory is not persisted (rebuilt from source files on startup). Snapshot size for 10,000 LTM entries is ∼3.2 MB; 50-entry preference sets are ∼14 KB.

3 Tiered SLM Router

3.1 Architecture

The router is a two-tier pipeline. Tier 1 uses FunctionGemma-270M (Google DeepMind, 2024), a RecurrentGemma-based function-calling specialist, to classify queries into one of four memory routes. Tier 2 falls back to Qwen3.5-0.8B (Bai et al., 2023) if Tier 1 fails to produce a parseable tool call.

3.2 FunctionGemma Float16 Silent Failure

During integration we discovered a previously undescribed failure mode. Loaded with `float16` on Apple MPS, FunctionGemma generates only `<pad>` tokens (token ID 0) for all inputs, producing an empty string with no exception or warning.

Root cause. FunctionGemma uses the RecurrentGemma SSM architecture with gated linear recurrence layers. Under `float16` on MPS, activation overflow in these layers produces `NaN` logits; `argmax(NaN)` defaults to index 0, which is `<pad>`. No exception is raised because the computation completes without error it simply produces incorrect logits.

Fix. Load FunctionGemma with `dtype=torch.float32` on MPS. Memory cost increases from ∼550 MB to ∼1.1 GB, acceptable for a 270M-parameter model. `bfloat16` would avoid the overflow (wider exponent range) but is not supported by Apple MPS. The silent failure occurs on any device without `bfloat16` support (all Apple Silicon MPS, pre-Ampere NVIDIA GPUs).

3.3 Tool Description Tuning

After fixing the precision issue, two misroutes persisted: (1) user preference for TypeScript routed to Tree instead of LTM, (2) "What is the capital of France?" routed to STM instead of Cloud.

Both were resolved by strengthening tool descriptions:

- **LTM**: Added explicit coverage: "*includes preferences for programming languages, tools, frameworks even if the preference mentions code names like TypeScript or Python. Do NOT use for code analysis or syntax fixing.*"
- **Cloud**: Added explicit negative: "*If the user never told you to remember something, do NOT route to STM or LTM route here instead.*"

No model weights were changed. Description quality is the primary routing accuracy lever.

3.4 Router Accuracy Evolution

Run 3's average latency (5,408 ms) exceeds Run 2's (3,255 ms) due to a cold-start first query (17,539 ms). Warm latency is 1,000–1,650 ms across both runs.

4 Single-File Validation: Inference Backend Comparison

Before introducing cross-file context injection, we establish the baseline performance of the Worker SLM on single-file analysis. This isolates the effect of inference backend from the effect of Tree Memory context, giving a clean reference for all subsequent multi-file experiments. We evaluate the same Qwen3.5-0.8B weights via two backends on two test files with 11 planted bugs (6 in `test_calculator.py`, 5 in `test_tictactoe.js`). No Tree Memory context is used in this experiment.

The gap is attributable to inference backend, not memory. Ollama's `qwen3.5:0.8b` runs in thinking mode on CPU, producing extended chain-of-thought loops that collapse into repetition before completing structured analysis. MPS inference with `do_sample=True, temperature=0.7` produces clean, structured output 4–5× faster. This backend difference is the starting condition for all multi-file experiments that follow.

5 Dependency-Aware Retrieval: Validation and Ablation

5.1 Motivation

Tree Memory stores project structure as a dependency graph and exposes import-scoped subgraph queries. The validation question is whether the retrieved context function signatures of files the current file imports conveys enough cross-file information to distinguish correct from incorrect cross-file usage. We use cross-file bug detection as a proxy: if a bug requires knowing a callee's type signature and Tree Memory provides

Table 2: Router accuracy and latency across evaluation runs (4 test queries).

Run	Configuration	Accuracy	Avg latency	Tier 1 active
Baseline	Qwen3.5-0.8B, regex matching	1/4 (25%)	11,762 ms	No
Run 1	FG `float16` (broken)	3/4 (75%)	9,770 ms	No (all `<pad>`)
Run 2	FG `float32`, old desc.	2/4 (50%)	3,255 ms	Yes
Run 3	FG `float32`, tuned desc.	4/4 (100%)	5,408 ms	Yes
Warm latency (Runs 2–3)			∼1,350 ms	

Table 3: Single-file analysis: TST (MPS) vs. Ollama (CPU), same Qwen3.5-0.8B weights, no Tree Memory.

Metric	Ollama (CPU)	TST (MPS)
Calculator bugs found (6)	∼4/6 (67%)	5/6 (83%)
TicTacToe bugs found (5)	∼4/5 (80%)	5/5 (100%)
Total	∼8/11 (73%)	10/11 (91%)
Hallucinations	Yes (3+ false positives)	0
Generation stability	Repetition collapse	Clean
Calculator latency	297,311 ms	63,804 ms
TicTacToe latency	105,578 ms	68,377 ms

Table 4: Controlled cross-file test, Run 1 (hint prompt).

Bug	TST (MPS)	Ollama (CPU)	Cross-file?
A (counts=0)	✓	✓	No
B (ZeroDivisionError)	✓	×	No
C (`str`→`function_e`)	✓	✓*	Yes
Total	3/3	2/3	

Table 5: Controlled cross-file test, Run 2 (clean prompt).

Bug	TST (MPS)	Ollama (CPU)	Verdict
A (counts=0)	✓	✓	Both
B (ZeroDivisionError)	×	×	Neither (0.8B)
C (`str`→`func_e`)	✓	FP	TST ✓; Ollama FP
Total	2/3	2/3	

that signature, a capable consumer should detect it. We conduct a controlled 2-run experiment establishing the retrieval quality bound, then a 4-run ablation characterizing the retrieval *selectivity* requirement.

5.2 Controlled Cross-File Reasoning Test

Setup. We create a clean test project (`test_project_clean/`) with two files. `file1.py` imports `function_e` from `file2.py`. Three bugs are planted in `file1.py`:

- **Bug A**: `counts[item] = 0` on first occurrence (should be 1) visible in `file1.py` alone.
- **Bug B**: `ZeroDivisionError` when `len(scores) == 1` visible in `file1.py` alone.
- **Bug C**: `function_e(str(rec["score"]))` passes a string to a function declared `def function_e(n: int) -> float:` in `file2.py` detectable *only* with cross-file knowledge.

Both models receive only `file1.py` code. The TST side additionally receives a tree context header: *"file2.py exports: `def function_e(n: int) -> float`"*. The Ollama side receives the raw file with no cross-file context. Source files contain no `# BUG:` comments (all such comments were stripped before testing).

Run 1 (hint prompt). Prompt: "Find all bugs, type errors, and runtime exceptions. Pay attention to function calls that may have type mismatches."

*Ollama's detection of Bug C was not genuine cross-file reasoning. Ollama hallucinated `def function_e(val: str) -> float: return float(val)` and concluded `str()` was correct, then reversed direction based on the hint phrase "type mismatches." The scorer matched because the keyword "int" appeared elsewhere in the response (`threshold: int`), not from detecting the actual mismatch.

Run 2 (clean prompt). Prompt: "Find all bugs and errors in this code." (no type-mismatch hint)

Ollama hallucinated its own version of `function_e` accepting `str` and concluded the call was correct. TST correctly identified the type mismatch using the injected signature. Bug B was missed by both (the trimmed-list ZeroDivisionError requires multi-step reasoning that exceeds 0.8B capacity without explicit prompting).

Key finding: TST is 2/2 correct on Bug C across both runs. Ollama is 0/2 correct (one scoring artifact, one hallucination). The controlled experiment confirms the tree context injection advantage on genuine cross-file type errors.

6 Multi-File Stress Test: 4-Tier Bug Detection

6.1 Experimental Design

We construct a 10-file Python project (`stress_testing/`) with 24 planted bugs spanning four difficulty tiers:

- **Tier 1** (6 bugs): Single-file bugs detectable without any cross-file knowledge (e.g., missing validation, incorrect status check).
- **Tier 2** (6 bugs): Signature mismatch bugs requiring the callee's function signature from another file (e.g., wrong argument count, wrong return type usage).
- **Tier 3** (4 bugs): Semantic contract bugs requiring documented behavioral contracts of imported functions (e.g., `acquire()` returns `None` when pool is exhausted).
- **Tier 4** (6 bugs): System-level bugs requiring understanding multi-file interaction patterns (e.g., sender debited before receiver existence check).

All source files were stripped of inline `# BUG` comments before evaluation. Scoring uses keyword matching: a bug is FOUND if all keywords in its set appear in the model response; PARTIAL if at least half appear; MISSED otherwise. TST (Qwen3.5-0.8B via HuggingFace, MPS) is evaluated under three context configurations. A separately run Ollama baseline (qwen3.5:0.8b) was found to have received uncleaned source files with inline bug annotations, making direct TST vs. Ollama comparison at Tiers 2–3 unreliable (see Limitations). The ablation therefore focuses on TST-only context format comparisons.

6.2 Retrieval Selectivity

Table 6 summarizes the three TST runs. Import-graph-scoped injection ($\sim$100–200 tokens, direct dependencies only) is the correct operating point: it recovers Tier 2 cross-file detections and reduces hallucinations 6-fold relative to bulk injection.

Run 1 (bulk injection). TST injects function signatures from all 9 other project files as a single prefix ($\sim$500–800 tokens) with greedy decoding (`do_sample=False`). Overall detection drops to 9.1%. Root cause: the long prefix exhausts the 0.8B model's effective attention capacity before it can reason about the file under analysis. Greedy decoding compounds this `do_sample=False` with long inputs triggers repetition collapse. Hallucinations averaged 0.6 per file. The Tier 2 matches in Run 1 are coincidental (keyword overlap), not genuine cross-file detections.

Run 2 (targeted, import-graph-scoped). `build_tree_context()` is rewritten to parse import statements statically and inject signatures *only* from files the current file directly imports ($\sim$100–200 tokens). Sampling is enabled: `do_sample=True, temperature=0.7, top_p=0.9`. The resulting dependency map:

```
models/user.py        -> []         (no
    project imports)
db/queries.py         -> [db/connection.py]
services/auth.py      -> [models/user.py,
    utils/crypto.py, db/queries.py]
services/payment.py   -> [models/user.py,
    models/transaction.py,
                          services/auth.py,
                              db/queries.py]
app.py                -> [db/connection.py,
    db/queries.py, services/auth.py,
                          services/payment.py
                              , services/
                              notification.py
                              ,
                          models/transaction.
                              py]
```

Two Tier 2 bugs are correctly identified genuine cross-file detections:

- **T2-3** (`services/auth.py`): `change_password()` calls `update_user_balance()` instead of the password-update method. TST received `db/queries.py` method names in context and identified the wrong callee.
- **T2-4** (`services/payment.py`): `user.credit()` called with no positive-value guard. TST received `models/user.py` signatures in context, which lack a validation contract, and flagged the missing check.

Without injected context, the callee's interface is invisible from the caller file alone. Hallucinations fall to 0.1 per file. The Tier 1 improvement (0% $\rightarrow$ 16.7%) is attributable entirely to the sampling fix, independent of context injection.

6.3 Format Sensitivity

Run 3 tests whether replacing Python stubs with compact STM-style Preference payloads reduces token count while encoding the same information semantically:

```
# Python stub (Run 2):
def acquire(self) -> Optional[Connection]:

# STM evidence (Run 3):
acquire_returns: Optional[Connection] (w
    =0.88)
acquire_can_return_none: True (w=0.90)
acquire_exists: True (w=0.82)
```

The result is 0.0% detection across all tiers. The root cause is format mismatch: Qwen3.5-0.8B was pre-trained on natural language and Python source, so it parses native function signatures

Table 6: Context injection ablation: three TST runs over the 24-bug benchmark.

Run	Context type	Tokens	Overall	Tier 2	Halluc/file
Run 1	Bulk stubs (all files)	500–800	9.1%	33.3%*	0.6
Run 2	Targeted stubs (imports)	100–200	13.6%	33.3%	0.1
Run 3	STM evidence format	200–350	0.0%	0.0%	0.2

*Run 1 Tier 2 detections are coincidental; see text.

natively. The custom key-value schema requires learning a new encoding from the prompt header alone, which this model was not trained to do.

Counterintuitively, token count was not reduced. The STM format generates 3–4 entries per function (return type, each argument, existence flag) versus 1 line per function in stub format, increasing average latency by 17 s. The correct serialization pipeline is: *store* as Preference payloads (compact, weight-ranked, queryable); *inject* as Python stubs (native model vocabulary). This is a serialization concern, not a store design concern.

6.4 Model Capacity Boundary

Tier 3 and Tier 4 bugs score 0% across all three TST runs regardless of context format or selectivity. The capacity limits are:

- **Tier 3 (contract bugs)**: Detecting that `acquire()` can return `None` requires understanding a documented behavioral contract, not just a type signature. Even with the explicit `acquire_can_return_none: True` evidence entry, the 0.8B model failed to connect this fact to the missing null check.
- **Tier 4 (system-level bugs)**: Detecting that a sender is debited before the receiver's existence is confirmed requires reasoning across the full payment flow spanning multiple files. No single-file context can substitute for this reasoning.

These are model-capacity limits that apply equally to all 0.8B consumers regardless of context quality. An 8B+ model would be required to reliably detect Tier 3 and 4 bugs. The store is not the bottleneck: the same `TREE_QUERY` subgraph that is insufficient for a 0.8B consumer provides complete, correctly-scoped information for a larger one.

7 Kernel and System Benchmarks

7.1 Experimental Setup

All benchmarks were run on Apple MacBook Air (M2, 8 GB RAM). The Rust kernel was compiled with `cargo build -release` (Rust 2021 edition). Python inference used `torch.float16` on MPS for Qwen3.5-0.8B and `torch.float32` on MPS for FunctionGemma-270M (Wolf et al., 2020; Paszke et al., 2019).

Table 7: Rust kernel STDIO throughput (Apple M2, single-threaded client).

Operation	Throughput	Latency	Spec
Write (STM/LTM)	106,354 req/s	0.009 ms	<1 ms
Read (key)	137,744 req/s	0.007 ms	<5 ms

7.2 Kernel Throughput

The kernel exceeds the write spec by 111× and the read spec by 714×.

7.3 TST Scaling

O(k) lookup behavior is confirmed to 1M entries. Memory scales at ∼24 bytes/node (cache-friendly sequential layout) as designed, with prefix sharing yielding ∼4.6 nodes/key at 1M entries.

7.4 STM Ring Buffer

10,256 total writes (256-slot buffer, 10,000 overwrite operations). After saturation: hit rate 100%, stale hits 0, hash index size 256 (no orphaned entries), lookup p99 <1 μs. STM-to-LTM promotion: 20 entries boosted to promotion threshold, all 20 verified in LTM after promotion pass.

7.5 Tree Memory Scaling

Monorepo root-to-full-tree traversal (∼20 ms) meets the <20 ms specification. For focused single-file queries, latency is 2–3 ms independent of project size. Cycle detection (visited-set) correctly terminates A→B→C→A cycles in 3 μs.

Table 8: TST lookup latency (p95) at increasing key counts.

Entries	Lookup p95	Memory	Throughput	Status
1K	<1 μs	0.1 MB	5.0M ops/s	PASS
10K	1 μs	1.0 MB	4.5M ops/s	PASS
50K	<1 μs	5.2 MB	5.3M ops/s	PASS
100K	1 μs	10.3 MB	5.8M ops/s	PASS
200K	4 μs	20.6 MB	7.6M ops/s	PASS (<25 MB spec ✓)
500K	6 μs	51.6 MB	8.4M ops/s	PASS
1M	9 μs	103 MB	8.0M ops/s	PASS (<1 ms spec ✓)

Table 9: Tree Memory scaling to monorepo size.

Scale	Nodes	Build	Query (root)	Memory
Small (20 files)	121	<1 ms	18 μs	12.6 KB
Medium (200 files)	2,201	<1 ms	215 μs	228 KB
Large (2k files)	22,001	2 ms	2,565 μs	2.3 MB
Monorepo (10k files)	110,001	19 ms	20,275 μs	11.4 MB

7.6 Persistence

50 preference payloads written to LTM, snapshot saved (14.3 KB), kernel restarted, snapshot loaded: cross-session recall 50/50 (100%). Snapshot save: 1,063 μs. Snapshot load: 88 μs. For 10,000 entries, snapshot size is 3.2 MB; load time <1 s. Partial corruption test (last 128 bytes truncated): `load_snapshot()` returns `Err` gracefully without panic.

7.7 Logit Bias Safety

The bias formula saturates naturally at $\ln(101) \approx 4.605$ for $f \geq 100$. Clamp bounds of ± 0.5, ± 1.5, ± 5.0, and ± 10.0 were tested: no `NaN`, no out-of-bounds output. Non-finite inputs (`NaN`, $\pm\infty$) return $b = 0$ after a guard added during stress testing. Formula is self-limiting: ± 5 and ± 10 clamps are functionally identical for this formula.

7.8 Stress Test Summary

8 Architectural Findings

8.1 Store Design Is Validated Independently of Consumer Quality

Tree Memory's scaling results (Table 9) are independent of the Worker SLM: 110K nodes at 11.4 MB, 19 ms build, 2–3 ms focused queries, with a stable STDIO API. A larger consumer 8B, 70B, or a static analyzer issues the same `TREE_QUERY` calls and receives the same correctly-scoped subgraph. The ablation experiments characterize what a 0.8B *consumer* can do with that store; they say nothing about the store's ceiling.

8.2 Import-Graph Scoping Restores Retrieval Quality

`TREE_QUERY(node_id, depth)` already fetches only the subgraph reachable from the current node. The initial bulk injection treated all 9 project files as relevant context for every file an inadvertent bypass of the store's selectivity guarantee. Restoring the import-graph boundary (what the store was designed to provide) recovered the Tier 2 advantage. The design is self-evidently correct in retrospect: a retrieval index that returns everything is not providing retrieval.

8.3 Serialization Layer Is Separate from Store Format

The STM `Preference` payload (key, value, weight) is an appropriate *internal* storage format. It is not appropriate as a *prompt injection* format for 0.8B models. The correct pipeline: *store* as Preference payloads in LTM/STM; *serialize* to Python stub syntax before injection into model context. For larger models that can parse novel schema instructions from a system prompt, the weight-annotated format may be preferable a model-scale-dependent choice, not a store design choice.

8.4 Sampling Requirement for Long-Context 0.8B Inference

`do_sample=False` (greedy decoding) produces repetition collapse on 0.8B models with context-length prompts. The failure is deterministic and silent: coherent opening tokens, then clause-level repetition until `max_new_tokens` is exhausted. `temperature=0.7, top_p=0.9` consistently produces structured output. For any production deployment of sub-1B models

Table 10: Rust stress test suite results (36/36 pass, Apple M2).

Test	Scale	Key metric	Status
2.1 TST Density	1M entries	9 μs p95	PASS
2.2 Pathological keys	4 distributions	$<1.25\times$ baseline	PASS
2.3 Arena churn	1M ops	<1 μs p99	PASS
2.5 Decay math	analytical	exact match	PASS
3.1 STM saturation	10,256 writes	100% hit rate	PASS
4.2 ACL enforcement	adversarial	100% reject	PASS
5.1 Tree scaling	110K nodes	20 ms query	PASS
5.2 Cyclic deps	A→B→C→A	3 μs	PASS
6.1 500-turn session	500 turns	no degradation	PASS
8.1 Bias formula	full range	no NaN	PASS
9.1 Persistence	50 preferences	100% recall	PASS

with context injection, sampling must be enabled.

9 Limitations

Benchmark latency measurements. Rust stress benchmarks discard lookup results with `let _ = result`. In release mode, the compiler may eliminate dead-code paths, meaning sub-100 ns figures may measure `Instant::now()` overhead rather than actual memory traversal. Production measurements should use `std::hint::black_box` and the `criterion` crate for statistically rigorous benchmarking.

Persistence is snapshot-based. The current implementation saves a full snapshot on demand; operations since the last snapshot are lost on crash. True write-ahead log (WAL) replay for operation-level crash recovery is not implemented.

Arena monotonic growth. Deleted TST nodes are tombstoned (payload cleared) but arena slots are not freed. Long-lived systems with high churn will accumulate tombstoned nodes. This does not affect lookup correctness or latency but inflates memory usage relative to active entry count.

Single-run variance. Bug detection results from single runs with `temperature=0.7` carry high variance. The same model at the same temperature found Bug B (ZeroDivisionError) in Run 1 and missed it in Run 2 of the cross-file test. Three or more runs per file are needed for reliable tier-level statistics.

Ollama baseline excluded. The Ollama stress test run used source files that still contained inline `# BUG` comments, making TST vs. Ollama comparison at Tier 2 and above unreliable. The ablation in Section 6 is therefore TST-only. A controlled Ollama re-run on clean files would be required to establish a valid comparison baseline.

0.8B model ceiling. Tiers 3 and 4 consistently score 0% for TST (targeted context, clean files). These bugs require multi-step reasoning over behavioral contracts and multi-file interaction patterns. Context quality cannot compensate for model capacity at this scale. An 8B+ model would be required to reliably detect these bugs.

10 Related Work

Memory-augmented LLMs. MemGPT (Packer et al., 2023) models an OS-style memory hierarchy with main and external context, but operates at message-page granularity and is designed for large-model backends. MemoryBank (Zhong et al., 2023) persists conversational summaries but lacks sub-millisecond retrieval and code-structure awareness. ReadAgent (Lee et al., 2023) compresses long documents but does not provide persistent key-value storage. Our system targets a different regime: sub-1B models, token-level access, and Rust-speed retrieval.

Retrieval-augmented generation. RAG (Lewis et al., 2020) retrieves document chunks at query time. This works well for document corpora but introduces latency incompatible with interactive edge use and lacks the structured, weight-annotated preference model that enables logit bias injection at generation time. Memorizing Transformers (Wu et al., 2022) embed kNN retrieval inside the attention layer, requiring model modification. TST Memory is a separate kernel layer that requires no model changes.

Tool-calling and function routing. Toolformer (Schick et al., 2023) trains models to call APIs inline. FunctionGemma (Google DeepMind, 2024) is a 270M function-calling specialist trained with OpenAI-style tool schemas. Our router treats memory tiers as tools: STM, LTM, Tree, and Cloud are the four dispatch targets. The routing pipeline is a practical appli-

cation of function-calling at the edge, following lightweight multi-model routing patterns (Ashfaqbs, 2024).

State space models and precision issues. Griffin (De et al., 2024) and Mamba (Gu and Dao, 2023) demonstrate that gated linear recurrences match Transformer quality at reduced compute. FunctionGemma uses the RecurrentGemma SSM architecture. We document a previously unreported failure mode where SSM layers produce silent `<pad>` output under `float16` on Metal Performance Shaders (MPS).

Ternary search tries. The TST data structure (Bentley and Sedgewick, 1997) provides O(k) string operations for k-character keys with excellent cache behavior under prefix sharing. We demonstrate empirically that a 24-byte node layout achieves 1M-entry lookup at $<10\,\mu$s p95, covering the full vocabulary scale required for token frequency memory.

11 Conclusion

We presented the TST Memory System v3.1, a three-tier structured memory kernel for edge language models. The kernel STM ring buffer, LTM Ternary Search Trie, and Tree Memory DAG achieves 137,000 read operations per second at $<$0.01 ms average latency within a 23 MB footprint for 200K symbols, validated across 36 stress tests. Tree Memory scales to monorepo size (110K nodes, 11.4 MB, 20 ms full traversal, 2–3 ms focused queries) and is model-agnostic: the store exposes dependency-aware subgraph retrieval via a stable STDIO API that any consumer can query.

The tiered router achieves 4/4 routing accuracy at 1,350 ms warm latency after fixing a previously undescribed FunctionGemma silent failure under `float16` on Apple MPS and tuning tool descriptions, with no model retraining.

The retrieval ablation characterizes what 0.8B consumers require from Tree Memory. Import-graph-scoped retrieval ($\sim$150 tokens, direct dependencies only) is the correct operating point: it recovers Tier 2 cross-file detection and reduces hallucinations 6-fold relative to bulk injection. STM Preference payloads are the right *storage* format; Python stubs are the right *injection* format for 0.8B code models a serialization concern, not a store design concern. Tier 3 and Tier 4 failures are model-capacity limits that apply equally to all 0.8B consumers regardless of context quality; the store is not the bottleneck.

The architecture is complete. LTM persists preferences across sessions. STM provides transient conversational context at sub-millisecond latency. Tree Memory indexes project dependency structure and returns targeted subgraphs on demand. The 0.8B limitation is documented honestly. Plugging in a larger model consumer is a configuration change, not an architectural one.

Acknowledgements

Hardware provided by personal Apple Silicon devices. Models: FunctionGemma-270M (Google DeepMind), Qwen3.5-0.8B (Alibaba Cloud). All experiments run on Apple MacBook Air (M2, 8 GB RAM).

References

Ashfaqbs. TinyLLM-usecases: Lightweight multi-model routing use cases, 2024. https://github.com/Ashfaqbs/TinyLLM-usecases.

Jinze Bai et al. Qwen Technical Report. *arXiv:2309.16609*, 2023. https://arxiv.org/abs/2309.16609.

Jon Bentley and Robert Sedgewick. Fast algorithms for sorting and searching strings. In *Proceedings of the 8th ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 360–369, 1997.

Soham De et al. Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv:2402.19427*, 2024. https://arxiv.org/abs/2402.19427.

Google DeepMind. FunctionGemma: Function calling with Gemma, 2024. https://ai.google.dev/gemma/docs/functiongemma.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*, 2023. https://arxiv.org/abs/2312.00752.

Kuang-Huei Lee et al. ReadAgent: A system for human-like reading understanding. *arXiv:2402.09727*, 2023. https://arxiv.org/abs/2402.09727.

Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. https://arxiv.org/abs/2005.11401.

Charles Packer et al. MemGPT: Towards LLMs as operating systems. *arXiv:2310.08560*, 2023. https://arxiv.org/abs/2310.08560.

Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. https://arxiv.org/abs/1912.01703.

Timo Schick et al. Toolformer: Language models can teach themselves to use tools. *arXiv:2302.04761*, 2023. https://arxiv.org/abs/2302.04761.

Thomas Wolf et al. Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP: System Demonstrations*, pages 38–45, 2020. https://arxiv.org/abs/1910.03771.

Yuhuai Wu et al. Memorizing transformers. In *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2203.08913.

Wanjun Zhong et al. MemoryBank: Enhancing large language models with long-term memory. *arXiv:2305.10250*, 2023. https://arxiv.org/abs/2305.10250.